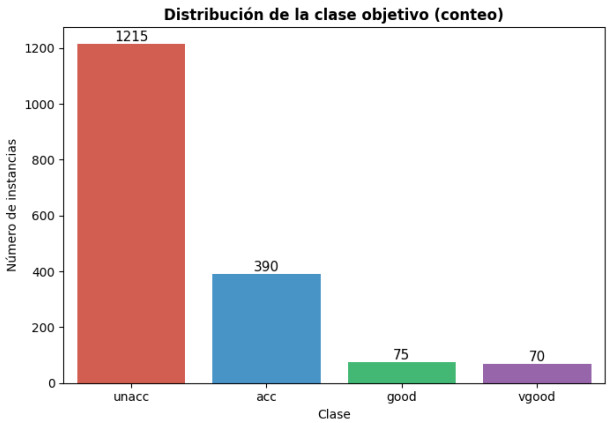
1. Descripción de la fuente de datos empleada

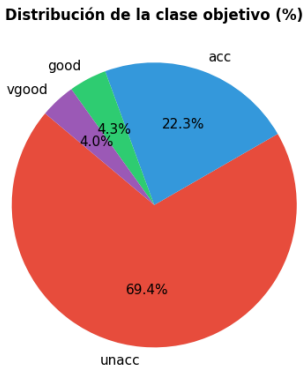
El dataset utilizado en esta actividad es el “Car Evaluation Dataset”, disponible públicamente en el repositorio UCI Machine Learning Repository (Bohanec, 1997). Contiene vehículos categorizados según parámetros de compra y características técnicas, con el objetivo de predecir su evaluación general. El dataset se carga prioritariamente desde el directorio de ejecución; si el archivo no se encuentra, el código recurre automáticamente a un repositorio público como fallback, garantizando reproducibilidad sin depender de rutas absolutas.

2. Caracterización del Dataset

El dataset contiene 1.750 instancias sin valores de atributo desconocidos, descritas mediante 6 atributos de entrada de naturaleza categórica ordinal. La variable objetivo es *class*, que representa la aceptabilidad (evaluación general) del vehículo y toma cuatro valores discretos. La distribución de clases es marcadamente desbalanceada: la clase *unacc* concentra el 69.4% (1,215 instancias), *acc* el 22.3% (390 instancias), mientras que *good* y *vgood* representan únicamente el 4.3% (75 instancias) y 4.0% (70 instancias) respectivamente. Este desbalance constituye un factor determinante en la estrategia de modelación adoptada.

Atributo	Descripción	Tipo	Valores posibles
Buying	Precio de compra del vehículo	Categórico ordinal	low, med, high, vhigh
Maintenance	Coste de mantenimiento	Categórico ordinal	low, med, high, vhigh
Doors	Numero de puertas	Categórico ordinal	2, 3, 4, 5more
Person	Capacidad de personas	Categórico ordinal	2, 4, more
lug_boot	Tamaño del maletero	Categórico ordinal	small, med, big
safety	Nivel de seguridad	Categórico ordinal	low, med, high
class (objetivo)	**Aceptabilidad del vehículo**	**Categórico ordinal**	**unacc, acc, good, vgood**





Principales hallazgos del Análisis Exploratorio de Datos (EDA)

El análisis exploratorio revela que todos los atributos de entrada presentan una distribución uniforme entre sus categorías, lo que garantiza que el método de selección de atributos basado en el índice Gini tenga suficiente variabilidad para discriminar clases. El atributo *safety* tiene un poder discriminativo muy alto: ningún vehículo con *safety=low* pertenece a las clases positivas (*good* o *vgood*). De igual manera, el atributo *Person* muestra que ningún vehículo con capacidad de 2 personas alcanza una clasificación positiva en el dataset, lo que refleja los supuestos del modelo de decisión original de Bohanec y no el comportamiento del mercado real. Ambos atributos producen subgrupos perfectamente puros (Gini=0.0), siendo candidatos fuertes para el nodo raíz. Como se verifica en el árbol inducido, CART seleccionó Person como nodo raíz por producir la mayor reducción de impureza global, con safety como segunda división.

El fuerte desbalance de clases (unacc=69.4%) representa el principal desafío de modelación. Sin corrección, cualquier clasificador tendería a predecir sistemáticamente la clase mayoritaria, obteniendo alta exactitud global pero un rendimiento muy bajo en las clases minoritarias *good* y *vgood*. Para compensar este efecto, se aplicó el parámetro *class_weight*='balanced' en ambos modelos, que asigna pesos inversamente proporcionales a la frecuencia de cada clase durante el entrenamiento.

3. Parámetros relevantes de los algoritmos

Modelo 1: Árbol de Decisión (CART)

CART (Pedregosa et al., 2011) utiliza el índice Gini como criterio de selección de atributos, a diferencia de ID3 que emplea ganancia de información basada en entropía (Tema 3, Ideas clave, UNIR). Parámetros configurados:

- criterion='gini': método de selección de atributos basado en el índice Gini, que mide la impureza de los datos en cada nodo. Determina cuál atributo produce la mejor división de las instancias.
- max_depth=6: profundidad máxima del árbol (poda preventiva). Limita la complejidad para evitar el sobreajuste, que ocurre cuando el árbol memoriza el ruido de los datos de entrenamiento.
- min_samples_split=10: número mínimo de instancias necesarias para dividir un nodo. Evita divisiones sobre subconjuntos muy pequeños que no son representativos.
- class_weight='balanced': asigna pesos inversamente proporcionales a la frecuencia de cada clase, compensando el fuerte desbalance del dataset.

Modelo 2: Random Forest (Aprendizaje Integrado - Bagging)

El Random Forest (ensemble learning, Bagging) construye árboles en paralelo sobre subconjuntos bootstrap, agregando predicciones por votación (Tema 3, Ideas clave, UNIR). Parámetros configurados:

- n_estimators=200: número de árboles de decisión en el bosque. Mayor número reduce la varianza, pero incrementa el coste computacional.

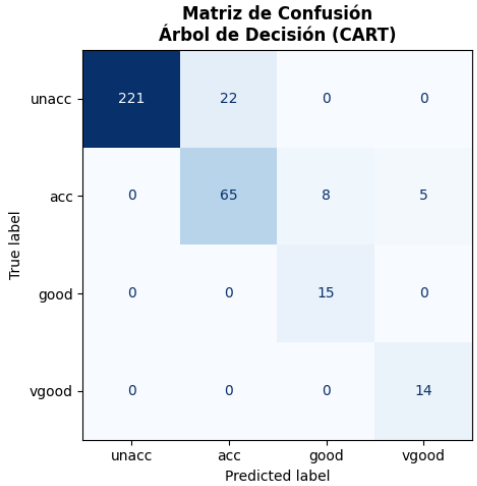
- max_depth=10: profundidad máxima de cada árbol individual del bosque.
- max_features='sqrt': en cada nodo solo se considera un subconjunto aleatorio de atributos de tamaño raíz cuadrada del total, introduciendo diversidad entre los árboles.
- class_weight='balanced': asigna pesos inversamente proporcionales a la frecuencia de cada clase, compensando el fuerte desbalance del dataset (unacc=69.4%).

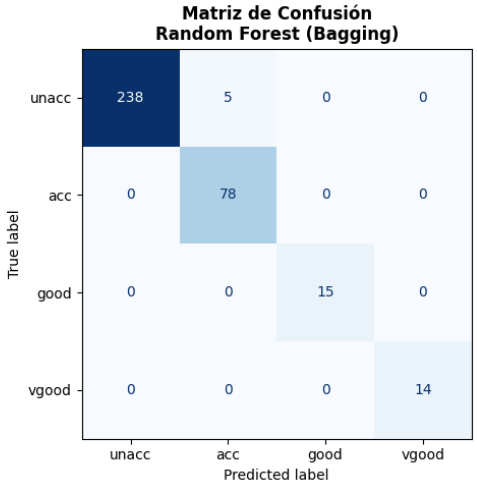
Estrategia de evaluación

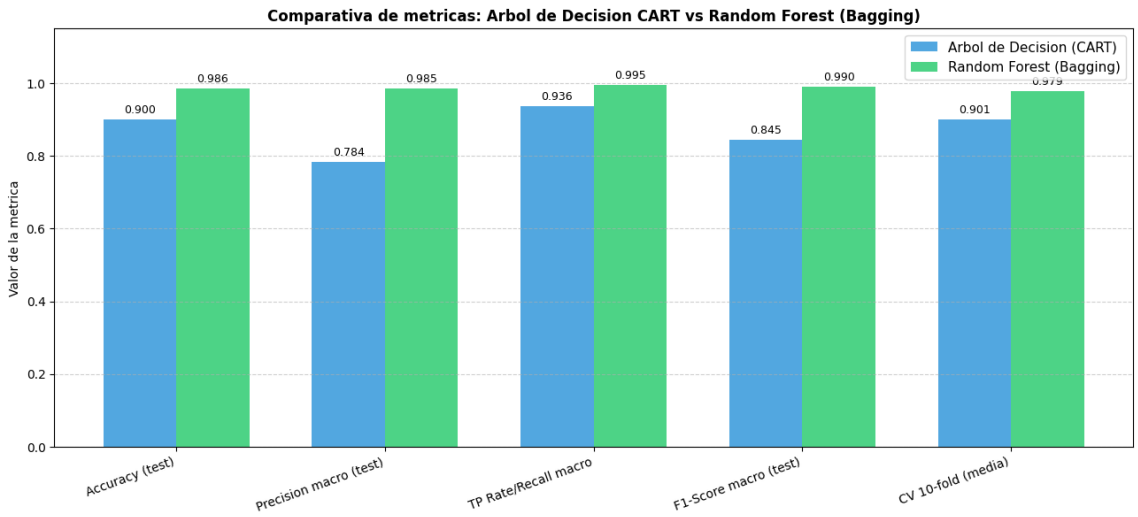
Partición hold-out estratificada 80/20: 1.400 instancias para entrenamiento y 350 para test. Se aplicó validación cruzada estratificada k=10 para estimar la generalización de los modelos.

4. Resultados comparados de los modelos

Métrica	Árbol de Decisión (CART)	Random Forest (Bagging)
Instancias clasificadas correctamente	315 / 350 (90.00%)	345 / 350 (98.57%)
Instancias clasificadas incorrectamente	35 / 350 (10.00%)	5 / 350 (1.43%)
TP Rate (Recall macro)	0.9357	0.9949
FP Rate promedio	0.0299	0.0046
Precisión macro	0.7840	0.9849
F1-Score macro	0.8446	0.9896







El Random Forest supera al Árbol de Decisión CART en todas las métricas evaluadas. La diferencia más pronunciada se observa en la Precisión macro (0.784 vs 0.985) y el F1-Score macro (0.845 vs 0.990), que ponderan por igual todas las clases. La desviación típica de la validación cruzada del Random Forest (0.0138) es notablemente menor que la del árbol individual (0.0299), lo que indica que el modelo integrado generaliza de forma más estable ante diferentes particiones de los datos. En términos de AUC — que toma valores entre 0.5 (clasificador aleatorio) y 1.0 (clasificación perfecta) (Tema 3, Ideas clave, UNIR) — Random Forest supera a CART en todas las clases, con la diferencia más significativa en acc (0.9652 vs 0.9993). Ambos modelos cometen errores exclusivamente en sentido ascendente (sobreestiman la aceptabilidad del vehículo), siendo este patrón de error más perjudicial en un contexto real que el caso contrario.

5. Discusión de resultados y propuestas de mejora

Ventajas y desventajas de cada modelo

CART destaca por su interpretabilidad: el árbol se traduce directamente en reglas legibles y no requiere conocimiento previo del dominio. Su limitación es la tendencia al sobreajuste; la poda preventiva (max_depth=6) la mitiga, pero introduce un sesgo que afecta la precisión en las clases minoritarias.

Random Forest resuelve el sobreajuste construyendo 200 árboles sobre subconjuntos bootstrap independientes; la agregación reduce la varianza sin incrementar el bias. Su desventaja es la pérdida de interpretabilidad: no produce reglas legibles, lo que dificulta la auditoría en contextos que requieren explicabilidad.

Propuestas de mejora

- Optimización de hiperparámetros: aplicar GridSearchCV con validación cruzada estratificada para encontrar la combinación óptima de max_depth, min_samples_split en CART y n_estimators, max_features en Random Forest, reduciendo el sesgo de la configuración manual.
- Tratamiento del desbalance de clases: técnicas de sobremuestreo como SMOTE (Chawla et al., 2002) sobre las clases good y vgood podrían mejorar el TP Rate de estas clases sin depender únicamente del ajuste de pesos.
- Algoritmos Boosting: AdaBoost o XGBoost entrenan árboles de forma secuencial priorizando las instancias peor clasificadas en cada iteración, lo que podría mejorar aún más el rendimiento en las clases minoritarias.
- Poda posterior C4.5: aplicar el algoritmo de poda pesimista de C4.5 (Quinlan, 1993) al árbol inducido por CART podría mejorar su capacidad de generalización sin necesidad de limitar su profundidad a priori.

• Bohanec, M. (1997). *Car Evaluation* [Dataset]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/19/car+evaluation
• Chawla, N. V., Bowyer, K. W., Hall, L. O., y Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, *16*, 321–357. https://doi.org/10.1613/jair.953
• Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://scikit-learn.org/stable
• Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. Morgan Kaufmann.
• UNIR. (2024). *Técnicas de inteligencia artificial: Ideas clave, Temas 1 y 3*. Universidad Internacional de La Rioja.